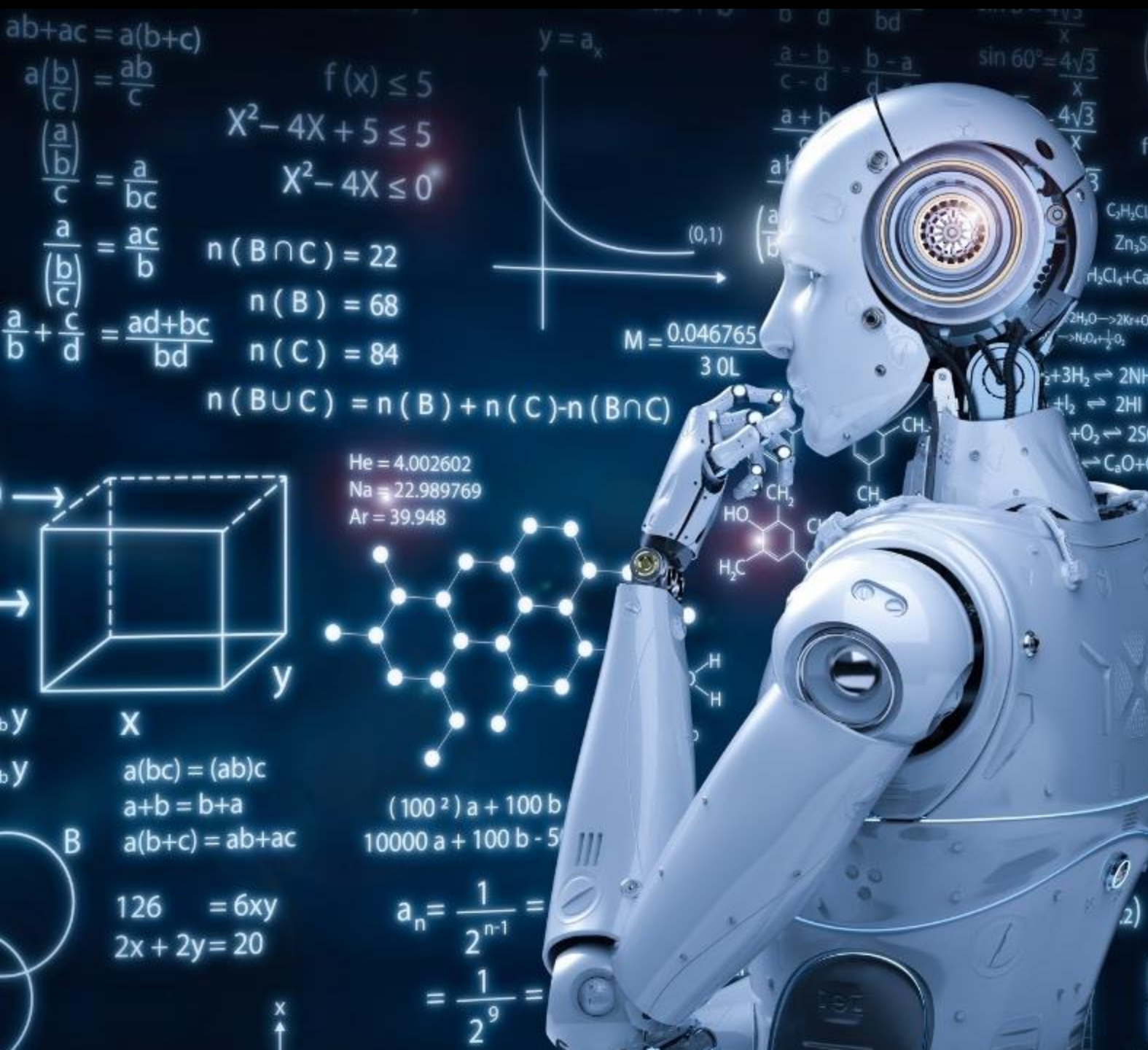




Learning Outcome 2

SEARCHING ALGORITHMS

PROF. DR. RASHMI GUPTA

RASHMI.GUPTA@KRISTIANIA.NO

Major Objectives

- Linear/Sequential Search
- Binary Search
- Binary Search Tree

Linear/Sequential Search

- ✓ Linear search is where we sequentially search the element in the sorted or unsorted list (usually an array).
 - ✓ **Sorted list:** Searching starts from the 0th element and continues until the element is found from the list or the element whose value is greater than (assuming the list is sorted in ascending order) the value being searched is reached.
 - ✓ $a = [1, 2, 3, 4, 5, 6, 7, 8, 12, 15]$ #sorted array
 - ✓ $B = [2, 1, 4, 10, 3, 5, 6, 22]$ #unsorted array
 - ✓ $X = 6, Z = 20$
 - ✓ **Unsorted list:** also begins from the 0th element and continues until the element or the end of the list is reached.

Linear/Sequential Search

- ✓ **Step 1:** First, read the search element (Target element) in the array.
- ✓ **Step 2:** In the second step, compare the search element with the first element in the array.
- ✓ **Step 3:** If both are matched, display “Target element is found” and terminate the Linear Search function.
- ✓ **Step 4:** If both are not matched, compare the search element with the next element in the array.
- ✓ **Step 5:** In this step, repeat steps 3 and 4 until the search (Target) element is compared with the last element of the array.
- ✓ **Step 6:** If the last element in the list does not match, the Linear Search Function will be terminated, and the message “Element is not found” will be displayed.



Understanding Complexity of Linear Search with Example

Task: Given an array `arr[]` of N elements, the task is to write a function to search a given element x_1, x_2, x_3, x_4 in `arr[]`.

Input: `arr[] = {10, 20, 80, 30, 60, 50, 110, 100, 130, 170}`,

$x_1 = 110; x_2 = 10; x_3 = 170; x_4 = 200$

- ✓ Every-Case Time Complexity $T(n) = n$
- ✓ Worst-Case Time Complexity $W(n) = n$
- ✓ Average-Case Time Complexity $A(n) = \frac{(n+1)}{2}$
- ✓ Best-Case Time Complexity $B(n) = 1$

Battle of Austerlitz “the finest battle I ever fought” – December 2, 1805 (Battle of the Three Emperors)

- ✓ Napoleon split the Austro-Russian Army and was able to conquer 2 weaker armies
- ✓ Divide an instance of a problem into 2 or more smaller instances
- ✓ Top-down approach





Binary Search

- ✓ Locate key x in an array of size n sorted in **non-decreasing order**
 - ✓ Compare x with the middle element (i.e., formula to calculate middle element $(\text{low} + \text{high}) / 2$)— if equal, done – quit. Else
 - ✓ **Divide the Array** into two sub-arrays approximately half as large
 - ✓ If x is smaller than the middle item, select the left sub-array
 - ✓ If x is larger than the middle item, select right sub-array
 - ✓ **Conquer (solve) the sub-array:** Is x in the sub-array using recursion until the sub-array is sufficiently small
 - ✓ **Obtain the solution** to the array from the solution to the subarray.
-
- ✓ **Rules:**
 - ✓ Array should be in **ascending order, descending order, or non-decreasing order.**
 - ✓ Calculate middle element
 - ✓ Compare the target element with middle element and decide what side of sub-array you choose.
 - ✓ Continue until you solve your problem.

Binary Search Algorithm

Algorithm 2.1

Binary Search (Recursive)

Problem: Determine whether x is in the sorted array S of size n .

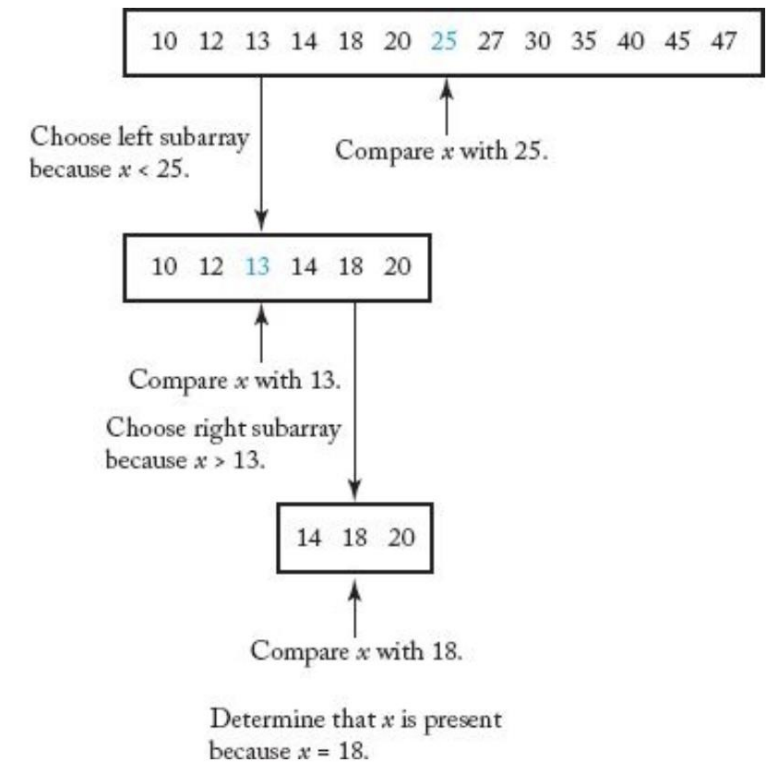
Inputs: positive integer n , sorted (nondecreasing order) array of keys S indexed from 1 to n , a key x .

Outputs: *location*, the location of x in S (0 if x is not in S).

```
index location (index low, index high)
{
    index mid;

    if (low > high)
        return 0;
    else {
        mid = ⌊(low + high)/2⌋;
        if (x == S[mid])
            return mid;
        else if (x < S[mid])
            return location(low, mid - 1);
        else
            return location(mid + 1, high);
    }
}
```

Figure 2.1 The steps done by a human when searching with Binary Search. (Note: $x = 18$.)



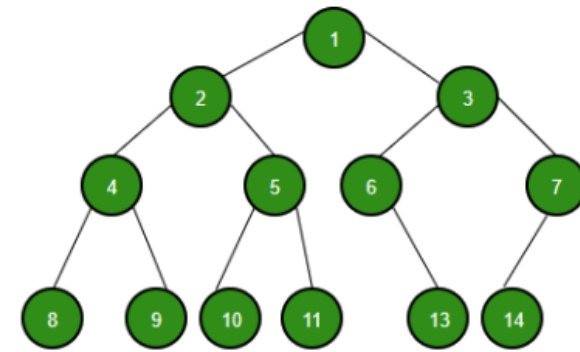
Pen-and-Paper Exercise

Use Binary Search, Recursive (Algorithm 2.1) to search for the **integer 120** in the following list (array) of integers. Show the actions step by step.

12 34 37 45 57 82 99 120 134

- ✓ Best-case Complexity $T(n) = O(1)$
- ✓ Average-case Complexity $T(n) = O(\log_2 n)$
- ✓ Worst-case Complexity $T(n) = O(\log_2 n)$
- ✓ Understand more of the Binary Search Complexity [Here](#).

Binary Search Tree



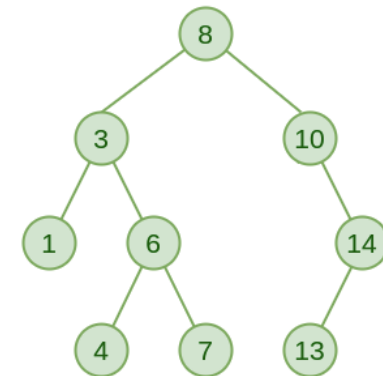
✓ First, what is Binary Tree?

✓ A tree data structure where each node has **at most** 2 children, typically name them the **left and right child**.

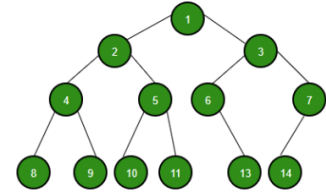
✓ Second, what is Binary Search Tree?

✓ A binary Search Tree is a **node-based** binary tree data structure which has the following properties:

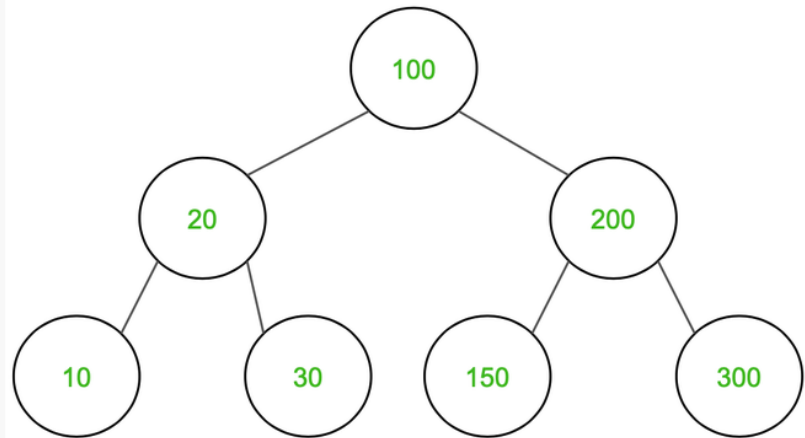
- ✓ The **left subtree** of a node contains only nodes with **keys lesser than the node's key**.
- ✓ The **right subtree** of a node contains only nodes with **keys greater than the node's key**.
- ✓ The **left and right subtree each must also be a binary search tree**.



Basic Operations: Binary Search Tree

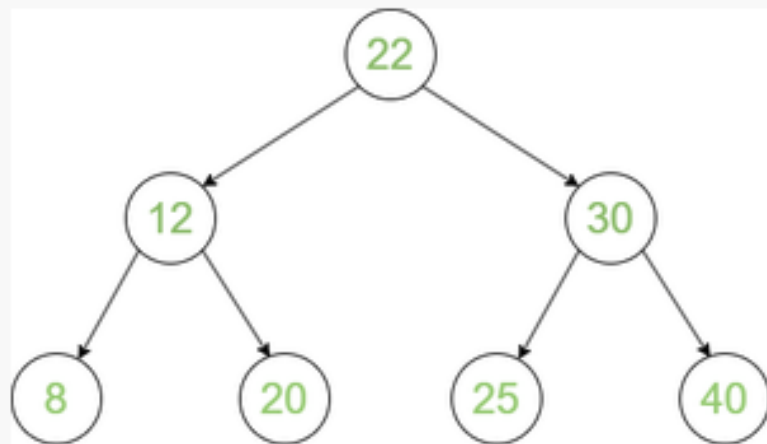


- ✓ Binary Search Tree (BST) **Traversals** – Inorder, Preorder, Post Order
- ✓ **Searching** in Binary Search Tree
- ✓ **Insertion** in Binary Search Tree
- ✓ **Deletion** in Binary Search Tree



Binary Search Tree

BST(a)



Binary Search Tree

BST(b)

Binary Search Tree (BST) Traversals

In-order, Pre-order, Post-Order

- ✓ The task is to print the elements in in-order, pre-order, and post-order traversal of the Binary Search Tree.
- ✓ **In-order Traversal:** Left-Root-Right
 - ✓ Output: 10 20 30 100 150 200 300
- ✓ **Pre-order Traversal:** Root-Left-Right
 - ✓ Output: 100 20 10 30 200 150 300
- ✓ **Post-order Traversal:** Left-Right-Root
 - ✓ Output: 10 30 20 150 300 200 100
- ✓ **Output of BST(b):**
 - ✓ **In-order Traversal:** 8 12 20 22 25 30 40
 - ✓ **Pre-order Traversal:** 22 12 8 20 30 25 40
 - ✓ **Post-order Traversal:** 8 20 12 25 40 30 22
- ✓ Resources to understand the **Java implementation of BST traversals** are [Here](#)
- ✓ Resources to understand the **Python implementation of BST traversals** are [Here](#).

Searching in Binary Search Tree

- ✓ Search operations in binary search trees is very similar to the binary search algorithm.
- ✓ For example, we want to search for the number,
 - ✓ start at the root,
 - ✓ compare the value to be searched with the value of the root,
 - ✓ if it's equal, we are done with the search
 - ✓ if it's smaller, we need to go to the left subtree because in a binary search tree, all the elements in the left subtree are smaller, and all the elements in the right subtree are larger.
- ✓ Searching an element in the binary search tree is basically the pre-order traversal, at each step, we go either left or right and at each step, we discard one of the sub-trees.
- ✓ The search here is also a binary search hence the name; Binary Search Tree.

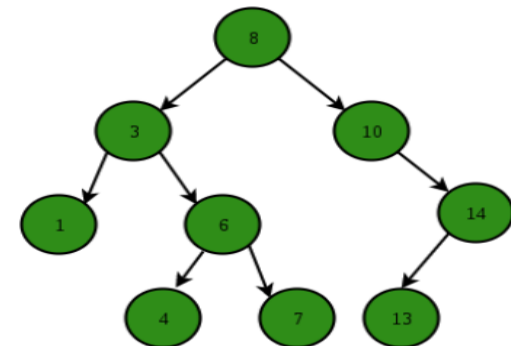
Java Implementation to Search a Given key in BST with Example

```
public Node search(Node root, int key)
{
    // Base Cases: root is null or key is present at root
    if (root==null || root.key==key)
        return root;

    // Key is greater than root's key
    if (root.key < key)
        return search(root.right, key);

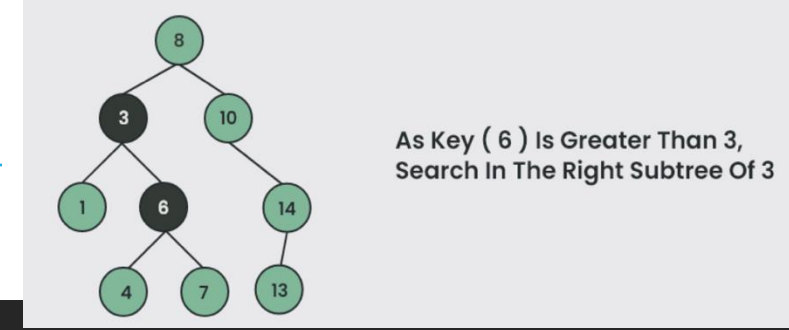
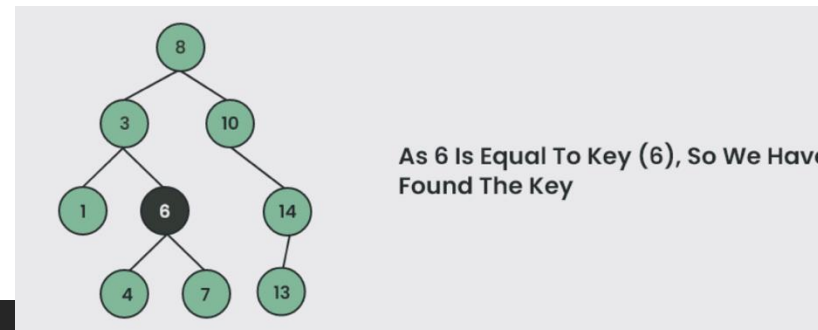
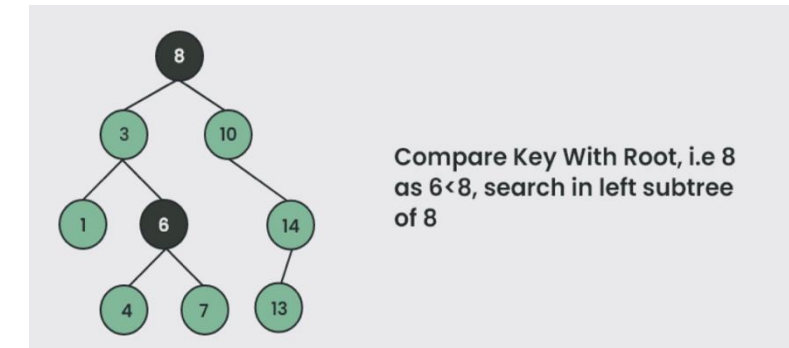
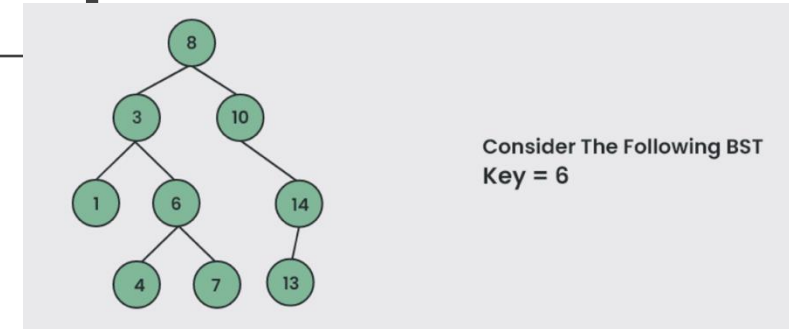
    // Key is smaller than root's key
    return search(root.left, key);
}
```

- ✓ Start from the root.
- ✓ Compare the searching element with root,
 - ✓ if less than root, then recursively call left subtree, else
 - ✓ recursively call right subtree.
- ✓ If the element to search is found anywhere, return true, else return false.
- ✓ Time complexity: $O(h)$, where h is the height of the BST.
- ✓ Space complexity: $O(h)$, where h is the height of the BST.
- ✓ This is because the maximum amount of space needed to store the recursion stack would be h .



Python Implementation to Search a Given key in BST with Example

- ✓ Start from the root.
- ✓ Compare the searching element with root,
 - ✓ if less than root, then recursively call left subtree, else
 - ✓ recursively call right subtree.
- ✓ If the element to search is found anywhere, return true, else return false.
- ✓ Time complexity: $O(h)$, where h is the height of the BST.
- ✓ Space complexity: $O(h)$, where h is the height of the BST.
- ✓ This is because the maximum amount of space needed to store the recursion stack would be h .
- ✓ You can understand the Python implementation of this algorithm [HERE](#).



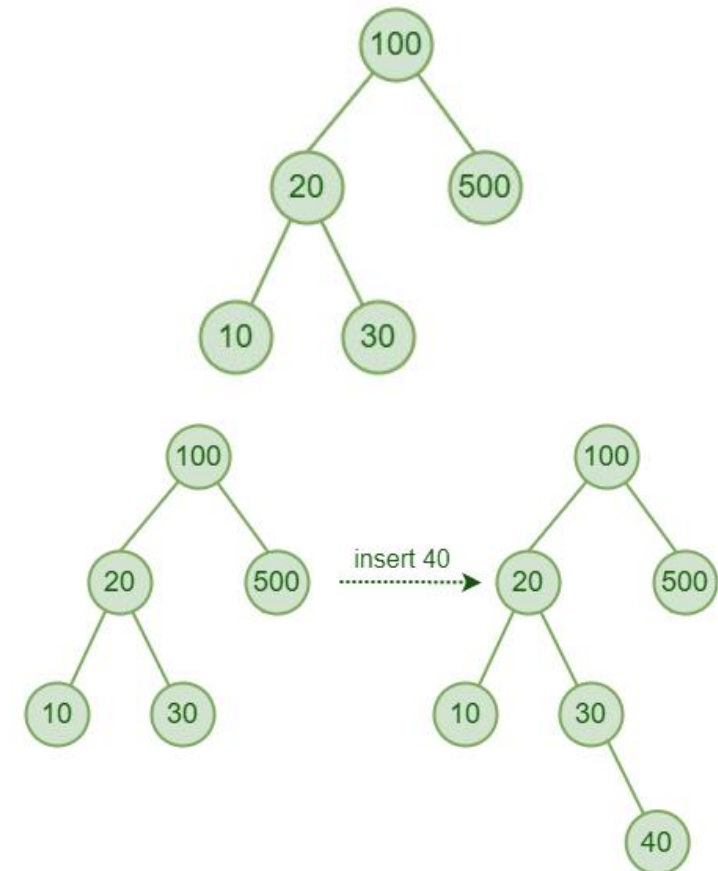
Insertion in Binary Search Tree

- ✓ A new key is always inserted at the leaf by maintaining the property of the binary search tree.
 - ✓ We start searching for a key from the root until we hit a leaf node.
 - ✓ Once a leaf node is found, the new node is added as a child of the leaf node.
- ✓ The below steps are followed while we try to **insert a node into a binary search tree**:
 - ✓ Check the **value to be inserted** (say **X**) with the **value of the current node** (say **val**) we are in:
 - ✓ If X is less than val move to the left subtree.
 - ✓ Otherwise, move to the right subtree.
 - ✓ Once the leaf node is reached, insert X to its right or left based on the relation between X and the leaf node's value.

Example: Insertion in Binary Search Tree

Let us try to insert a node with **value 40**, i.e., **X=40** in this tree:

- ✓ **1st step:** 40 will be compared with root, i.e., 100.
 - ✓ 40 is less than 100.
 - ✓ So move to the left subtree of 100. The root of the left subtree is 20.
- ✓ **2nd step:** 40 is now compared with 20.
 - ✓ It is greater than 20.
 - ✓ So move to the right subtree of 20 whose root is 30.
- ✓ **3rd step:** 30 is a leaf node.
 - ✓ So, we must insert 40 to the left or right of 30.
 - ✓ As 40 is greater than 30, insert 40 to the right of 30.
- ✓ Java implementation of this algorithm can be found [Here](#) for practice.
- ✓ Python implementation of this algorithm can be found [Here](#) for practice.

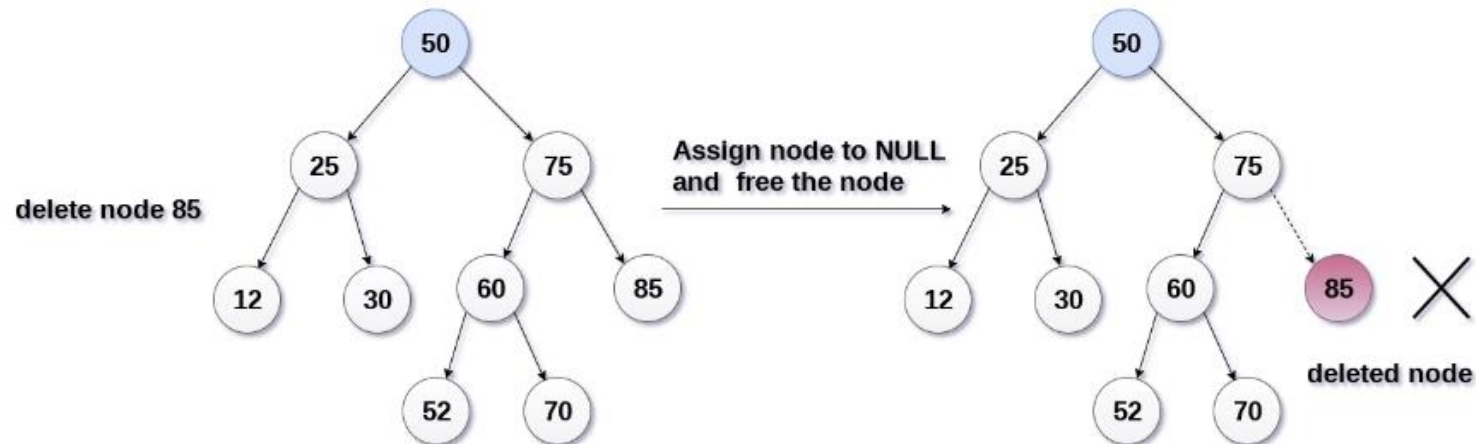


Deletion in Binary Search Tree

- ✓ When we delete a node, three possibilities arise.
 - ✓ **Node to be deleted is the leaf:** Simply remove it from the tree.
 - ✓ **Node to be deleted has only one child:** Copy the child to the node and delete the node.
 - ✓ **Node to be deleted has two children:** Find in-order successor of the node. Copy contents of the in-order successor to the node and delete the in-order successor.
 - ✓ Note: In-order predecessor can also be used.

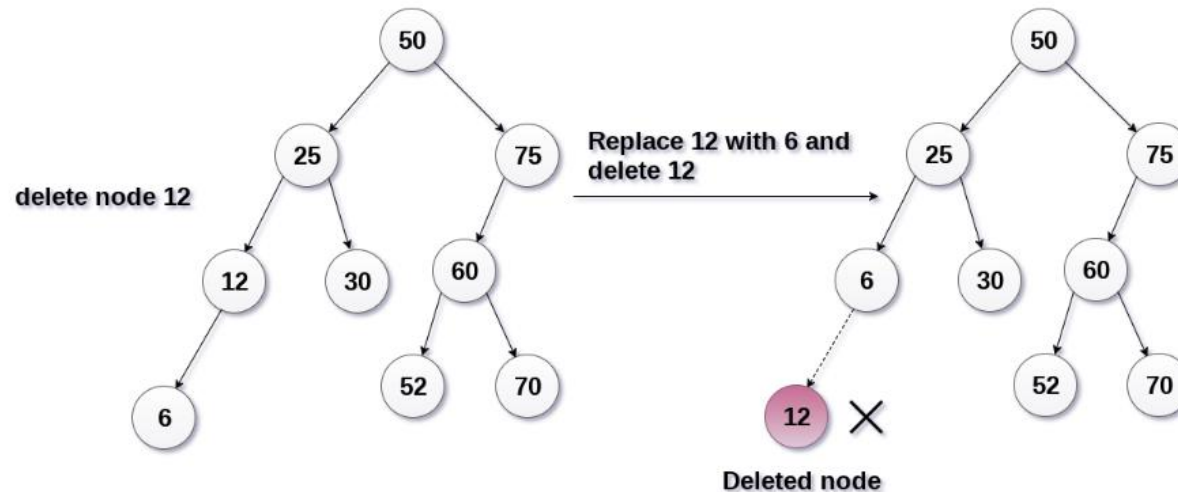
Node to be Deleted is the Leaf Node

- ✓ It is the simplest case, in this case, replace the leaf node with the NULL and simple free the allocated space.
- ✓ In the following image, we are deleting the node 85, since the node is a leaf node, therefore the node will be replaced with NULL and allocated space will be freed.

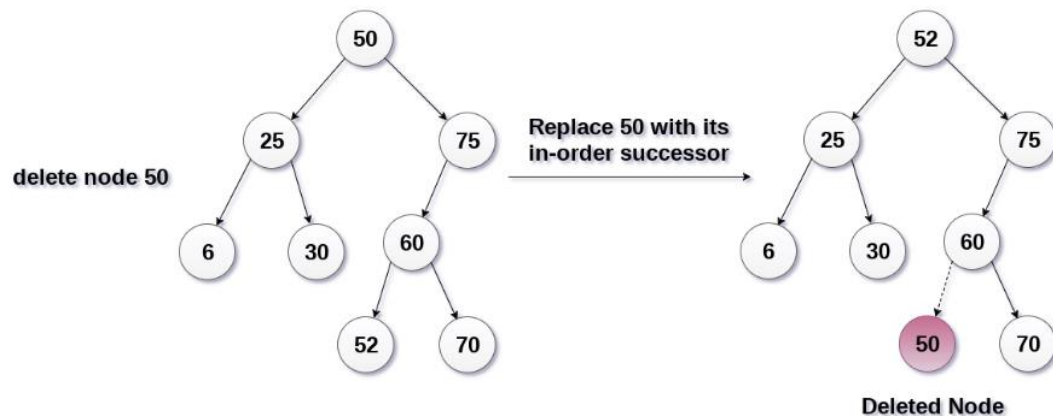


Node to be Deleted has Only One Child

- ✓ In this case, replace the node with its child and delete the child node, which now contains the value which is to be deleted. Simply replace it with the NULL and free the allocated space.
- ✓ For example, the node 12 is to be deleted in figure below. It has only one child. The node will be replaced with its child node, and the replaced node 12 (which is now leaf node) will simply be deleted.



Node to be Deleted has Two Children



- ✓ It is a bit complex case compared to the other two cases.
- ✓ The node which is to be deleted is replaced with its in-order successor or predecessor recursively until the node value (to be deleted) is placed on the leaf of the tree.
- ✓ After the procedure, replace the node with NULL and free the allocated space.
- ✓ For example, node 50 is to be deleted, which is the root node of the tree. The in-order traversal of the tree is given below.

In-order traversal: 6, 25, 30, 50, 52, 60, 70, 75

- ✓ replace 50 with its in-order successor, 52. Now, 50 will be moved to the leaf of the tree, which will simply be deleted.
- ✓ Java implementation can be found [HERE](#).
- ✓ Python implementation can be found [HERE](#).